UNIVERSITÀ di VERONA
Dipartimento di INFORMATICA

Discrete Optimization and Decision Making

Local Search

Master's degrees in Data Science - Artificial Intelligence - Mathematics
Academic Year 2024/2025

Romeo Rizzi (Dept. of Computer Science, Univ. of Verona)
Roberto Zanotti (Dept. of Clinical and Experimental Sciences, Univ. of Brescia)

Solution Algorithms for Integer Programming Problems: Local Search Heuristics

Heuristic Algorithms 1/2

From the greek word εὑρίσκω, *eurisko* → to find or discover.

Objective:

determining a good solution (not necessarily the optimal one) in a reasonable amount of time.

Properties:

- Computing time should not grow too rapidly when the size of the problem grows (i.e. time complexity should be a polynomial function in the size of the problem with low degree);
- Identified solutions should be, at least for the majority of the instances of the problem, optimal or close to the optimal one;
- It is a compromise between quality of the solution and time needed to find it.

Heuristic Algorithms 2/2

Minimum Problem

A heuristic A determines an **Upper Bound** on the optimal value:

$$z^* : z^A \geq z^*$$

(**Lower Bound** for a **maximum** problem).

Two main families of heuristic algorithms:

- **Constructive heuristic algorithms**
  - start from an empty solution;
  - iteratively determine new elements to add, until a complete (feasible) solution has been reached.
- **Local search algorithms**
  - start from a feasible solution x;
  - iteratively try to improve it by applying changes to the current solution (*moves* in a neighborhood of x);
  - terminate when there are no moves (of the selected type) that can improve the current solution (local minimum).

Examples of constructive algorithms: TSP 1/3

The Traveling Salesman Problem (TSP):

The constructive algorithms for the TSP start from an empty solution and iteratively determine the new vertices to add to the solution, until a complete solution has been reached:

1. Chose an arbitrary vertex and consider the corresponding self-loop as the initial partial solution S.
2. Expand S by inserting one by one the remaining vertices, finally obtaining an hamiltonian cycle.

   This procedure is done in two separate phases:

   a) select a vertex k, not included in S, according to some ***criterion***;
   b) insert vertex k in S, between two specific consecutive vertices u, v.

Examples of constructive algorithms: TSP 2/3

Several selection ***criteria*** can be used in point 2a:

- *Nearest neighbor*: select the vertex with the minimum distance from S. (average case: 20%; computational complexity $O(n^2)$)
- *Farthest insertion*: select the vertex with the maximum distance from S. (average case: 10%; computational complexity $O(n^2)$)
- *Arbitrary insertion*: randomly select the vertex to insert in S. (average case: 11%; computational complexity $O(n^2)$).
- *Cheapest insertion*: select the vertex with the minimum insertion cost. (average case: 17%; computational complexity $O(n^2 logn)$).

Examples of constructive algorithms: TSP 3/3

Point 2b: the vertex is usually inserted in a way that minimizes the "insertion cost":

- for each $(i, j) \in S$ we compute $d_{ij} = c_{ik} + c_{kj} - c_{ij}$;
- k is inserted between the consecutive vertices u, v such that:

$$d_{uv} \leq d_{ij} \qquad \forall (i, j) \in S.$$

Examples of constructive algorithms: 0–1 KP

0–1 Knapsack Problem

Greedy Algorithm:

1. Sort the items in non-increasing order of profit/weight ratio;
2. Add items in the knapsack until you reach the first one that exceeds the capacity.

Alternatively: sort the items in non-increasing order of profit or non-decreasing order of weight.

Algorithm performance analysis

How do you evaluate the quality of an algorithm?

There are two ways to analyze the performance of a heuristic algorithm:

1. *Experimental.* The quality of the solutions obtained for a benchmark set of instances is evaluated:
   - Can always be performed.
   - Not generalizable.
   - Performance are evaluated according to the solution quality/computing time ratio.
2. *Worst case.* The greatest error from the optimal solution is analytically computed:
   - Generalizable, but hard to determine.

Greedy Algorithm for the TSP

1. **Step 1.** Choose a starting node i and add it to the empty partial solution W, i.e. $W := \{i\}$. Set $r = i$.
2. **Step 2.** Select $s \in V \setminus W$ with minimum distance from node r, i.e. $d_{rs} = \min_{j \in V \setminus W} d_{rj}$, where d_{ij} is the length of arc (i, j). Insert r after s in W.
3. **Step 3.** Set $W := W \cup \{s\}$ and $r = s$. If $W = V$, stop. Node i is the one following node s, and W is now a cycle. If that is not the case, return to Step 2.

The heuristic has a polynomial computational complexity: the number of operations performed by the algorithm is $O(n^2)$, where $n = |V|$.

The quality of the solution can be terrible: the ratio between the identified solution and the optimal solution can be infinitely high. Let's see it in the next example.

Greedy Algorithm for the TSP: worst case example 1/2

Let us consider an undirected complete graph with 4 nodes. The following (symmetric) table shows the distances between each pair of nodes:

	1	2	3	4
1	-	2	M	2
2	2	-	2	1
3	M	2	-	2
4	2	1	2	-

Starting from node 1 (and similarly for all the other nodes), we obtain the hamiltonian cycle: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$ of length $5 + M$.

Greedy Algorithm for the TSP: worst case example 2/2

- For $M \leq 3$, the cycle identified by the heuristic is the optimal one, but for $M > 3$ the optimal solution becomes the hamiltonian cycle: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$, with objective function value equal to 8.
- For $M > 3$ the ratio between the objective function value of the cycle identified by the heuristic and the optimal one is:

$$\frac{5 + M}{8}$$

  and it grows to infinity for $M \rightarrow \infty$.

Neighborhood Function

- Given an optimization problem $P = (f, S)$
  - S set of all the feasible solutions for P;
  - $f : S \to R$ objective function;
- Neighborhood
  $N : S \to 2^{|S|}$ that $\forall i \in S$ defines $N(i) \subseteq S$ set of all the solutions close to i;
- Local Search algorithms try to improve a solution by exploring one of its neighborhoods.

Iterative Improvement Algorithms 1/3

First improvement: the neighbors of the current solution are analyzed in random order. A move is made towards the first one that improves the current solution (the algorithm exits the **for** cycle at the first improvement).

Procedure FI_Simple_Descent(s) /*$s \in S$ *initial sol.**/

```
Found := TRUE;
while Found = TRUE do
    Found := FALSE;
    for each s' ∈ N(s) do
        if f(s') < f(s) then
            s := s'; Found := TRUE; break;
end while
return (s);
```

Converges to a local optimum s with respect to $N(\cdot)$, i.e. a solution $s : f(s) \leq f(i)$ $\forall i \in N(s)$;

Iterative Improvement Algorithms 2/3

Best improvement: all the neighbors of the current solution are examined, and a move towards the best one is made. It is a *steepest descent* method: the move that produces the greatest improvement is made.

Procedure BI_Simple_Descent(s) /*$s \in S$ *initial sol.**/

```
Found := TRUE;
while Found = TRUE do
    Found := FALSE; s_best := s;
    for each s' ∈ N(s) do
        if f(s') < f(s_best) then
            s_best := s';
    if s_best ≠ s then
        s := s_best; Found := TRUE;
end while
return (s);
```

Iterative Improvement Algorithms 3/3

Main drawback of the Best Improvement algorithm:

Each iteration of the algorithm requires a lot of time: the time needed to evaluate $f(s)$ for $O(|N(s)|)$, where $|N(s)| :=$ is the cardinality of the neighborhood.

$$\Downarrow$$

the number of iterations required to reach the local optimum could be very high (although polynomial).

Local Search Algorithms

Advantages

- broad applicability;
- flexibility with respect to changes to the problem;
- can be used even when the solution is not feasible.

Disadvantages

- they cannot escape local minima, since they do not allow moves towards solutions that are worse than the current one.

Require

- A solution evaluator (objective function);
- feasibility check for a solution;
- neighborhood function;
- efficient neighborhood exploration technique.

Example 1:

2-opt heuristic

Traveling Salesman Problem TSP $(G(V, E), c, min)$.

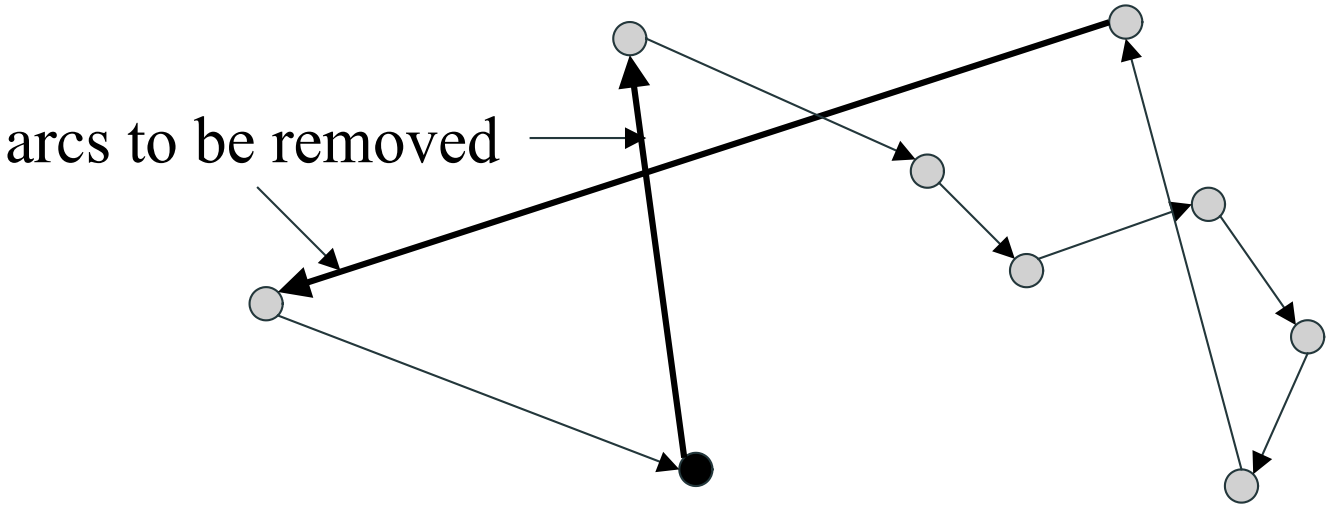
Basic idea: 2 arcs are removed at each step, then the two paths are reconnected with two different arcs.

2-opt algorithm:

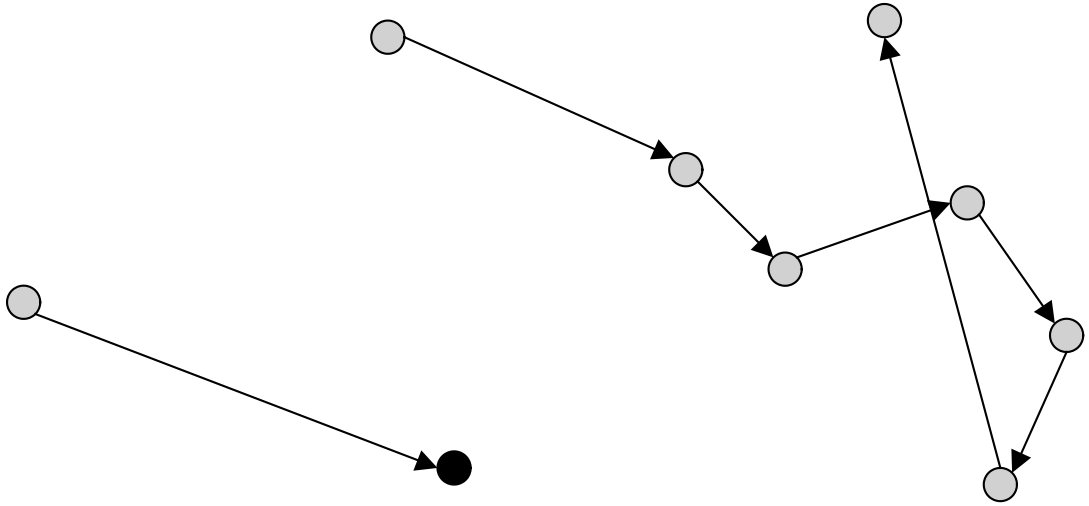
- start from an hamiltonian cycle,
- perform the 2-opt swaps between all pairs of arcs that reduce the circuit length.

Effectiveness: 8% more than the minimum, on average.

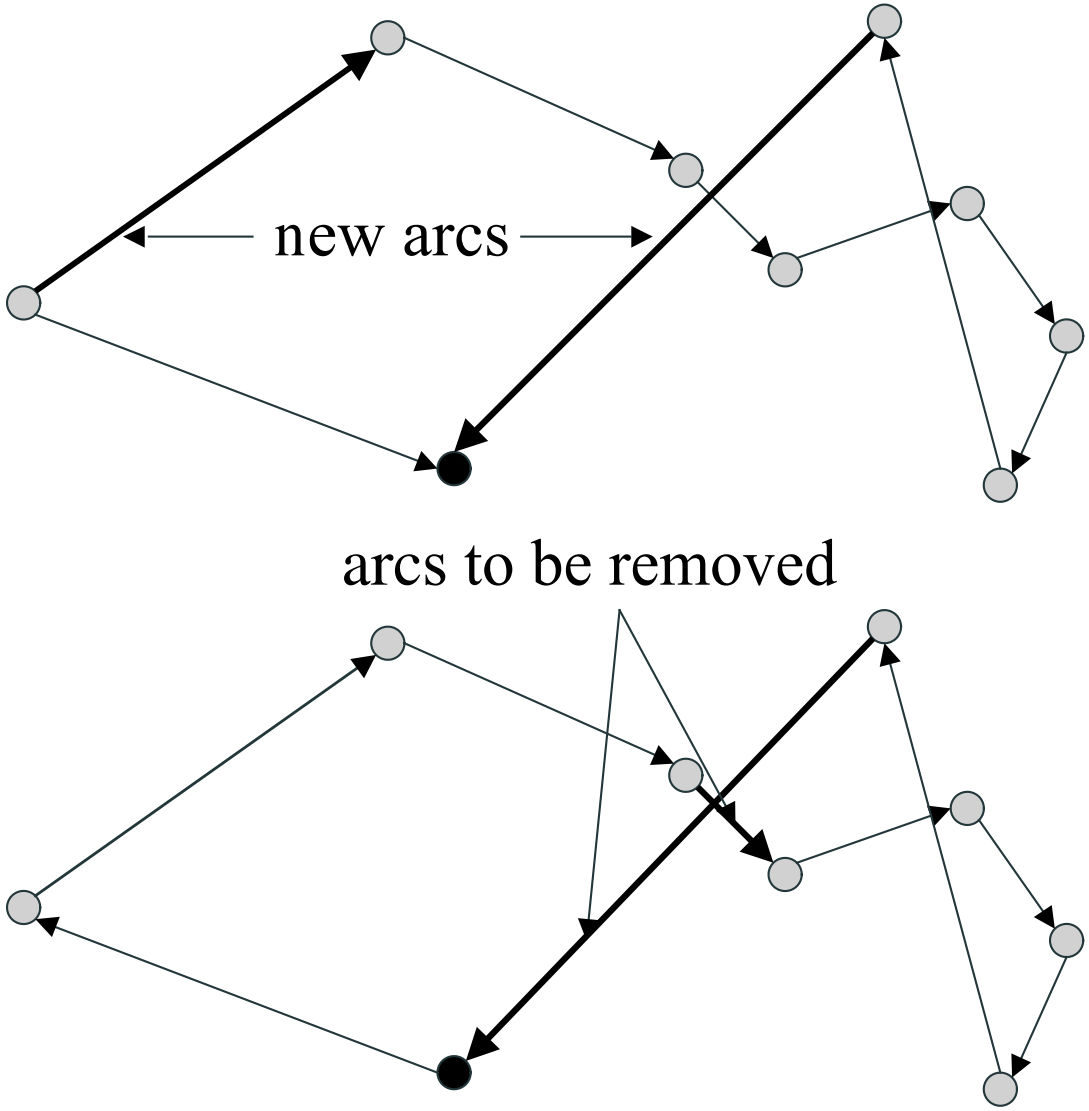
Initial cycle:



Paths:

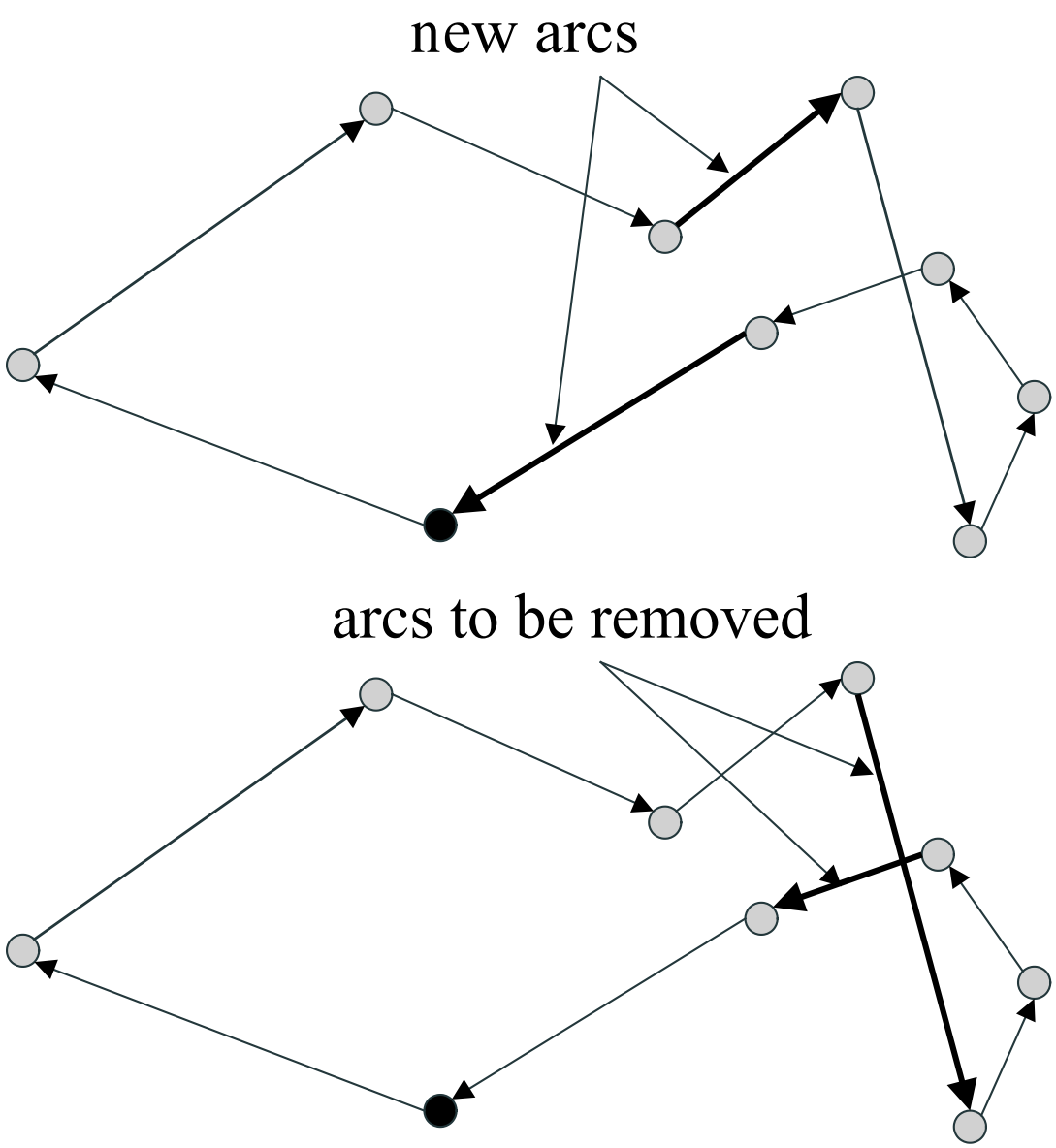


Cycle at Step 1:

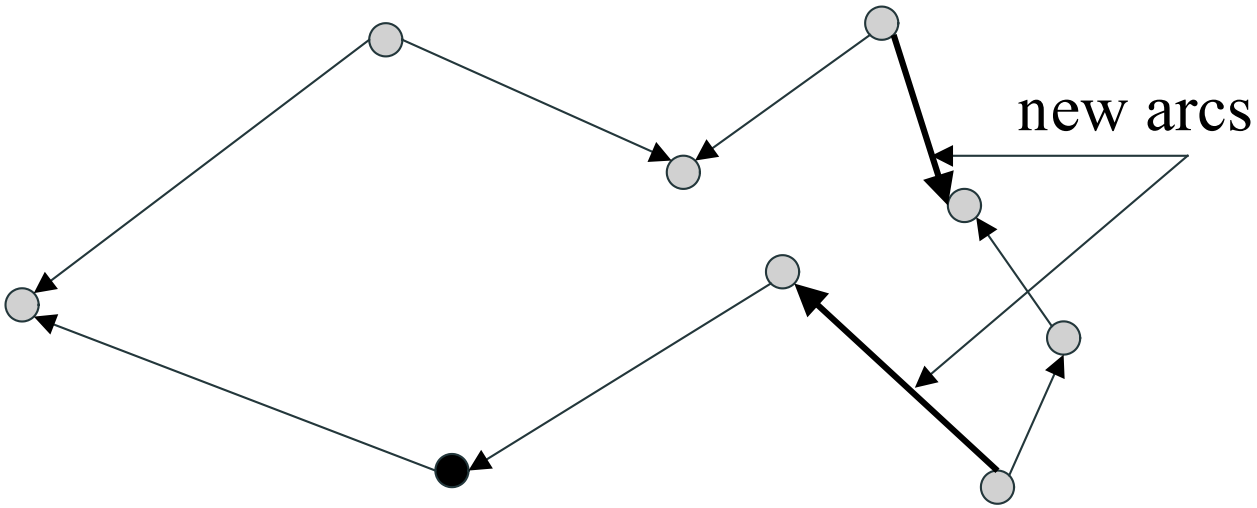


Notice the inversion of the direction of travel in one of the paths.

Cycle at Step 2:



Cycle at Step 3:



Notice the inversion of the direction of travel in one of the paths.

Example 2:

3-opt heuristic

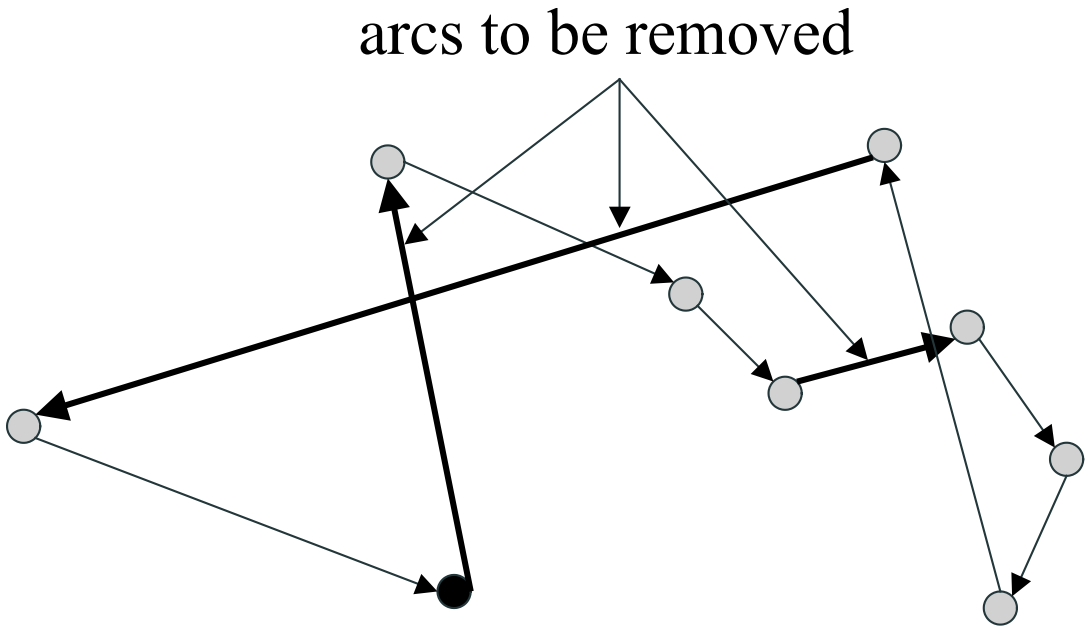
Traveling Salesman Problem TSP $(G(V, E), c, min)$.

Basic idea: 3 arcs are removed at each step, then the 3 paths are reconnected with 3 different arcs.

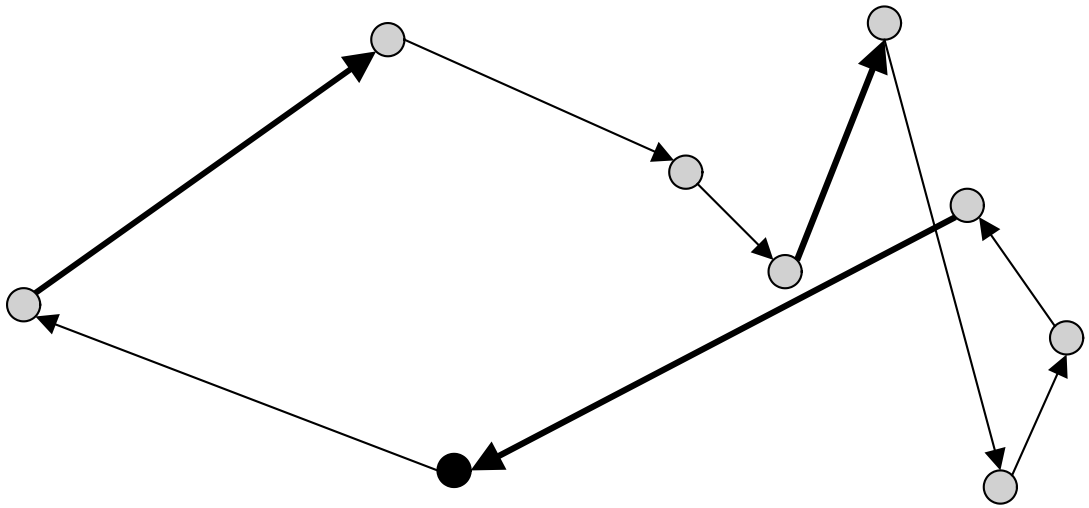
Effectiveness: 4% more than the minimum, on average.

The idea can be generalized up until the k-opt. For $k = n$ the neighborhood becomes exact and contains all the possible hamiltonian cycles, which are $n!$.

Initial cycle:



New cycle:



Meta-heuristic techniques

Problem:

Heuristic descent methods risk getting stuck in local optima that are not global optima.

Solution: meta-heuristic algorithms:

- local search algorithms that emply special techniques to escape local minima;
- algorithms that have to avoid cycles (repetition of already visited solutions).

Examples:

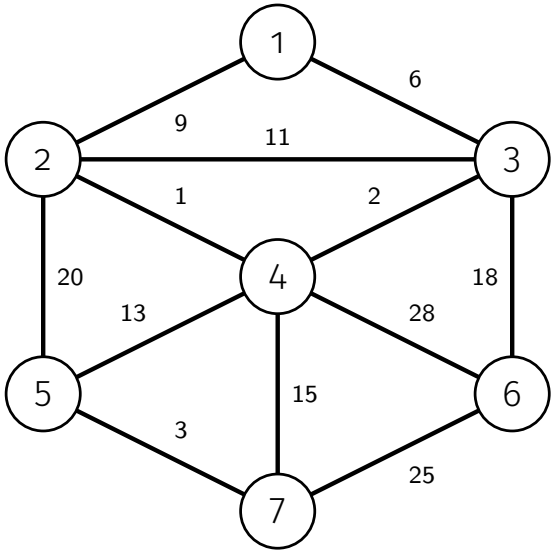
- **Simulated Annealing (SA)**
- **Tabu Search (TS):** allows a move towards a solution worse than the current one and avoids to visit already seen solutions by means of a list of forbidden (**tabu**) moves.
- **Genetic Algorithms (GA)**

Local Search Examples

Minimum Spanning Tree

$n = 7$ nodes

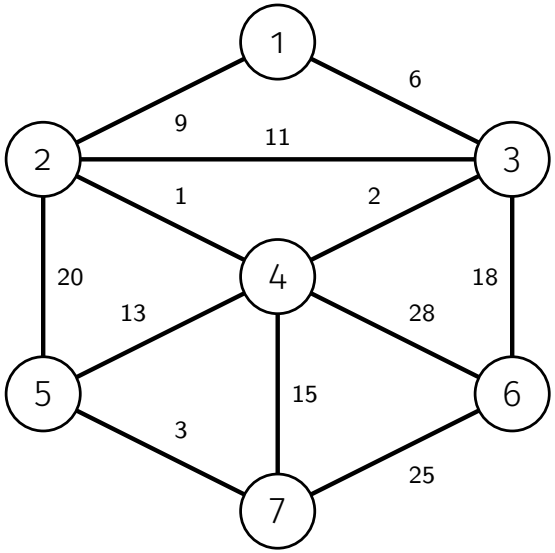
$m = 12$ edges

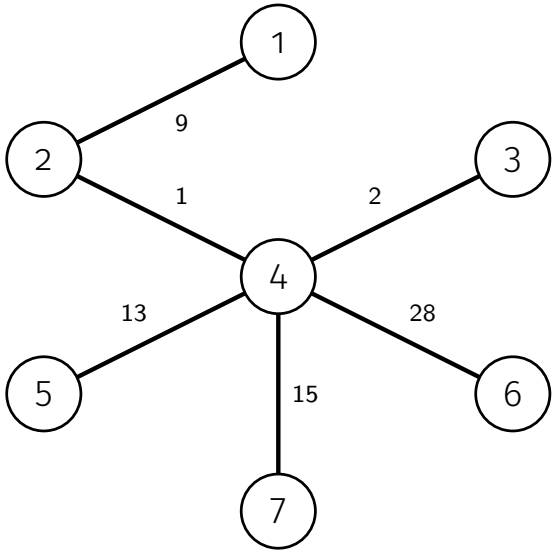


Minimum Spanning Tree

$n = 7$ nodes

$m = 12$ edges

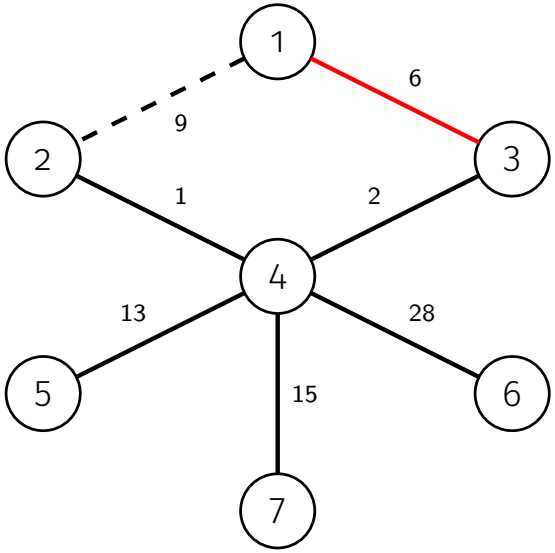




Sol = 68

Move definition

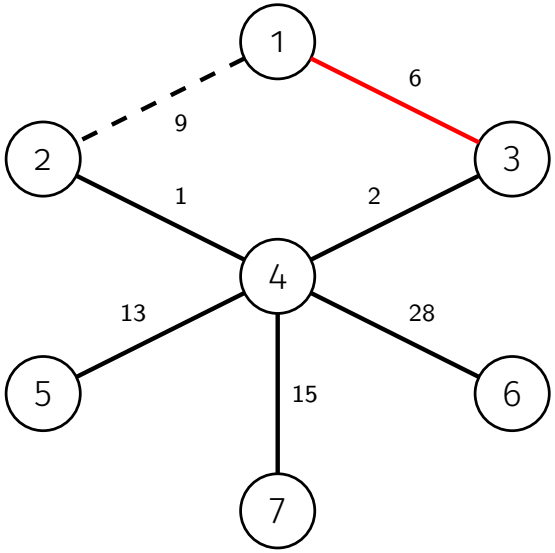
An edge is removed from the solution, while another one is added.
Note: the move has to keep the solution feasible.



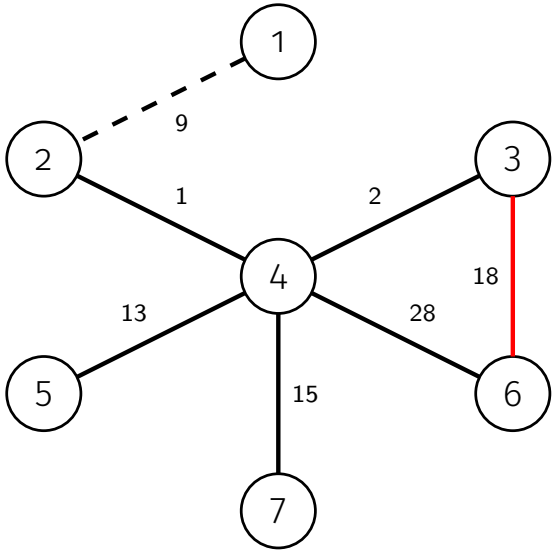
Sol = 68-9+6=65

Move definition

An edge is removed from the solution, while another one is added.
Note: the move has to keep the solution feasible.



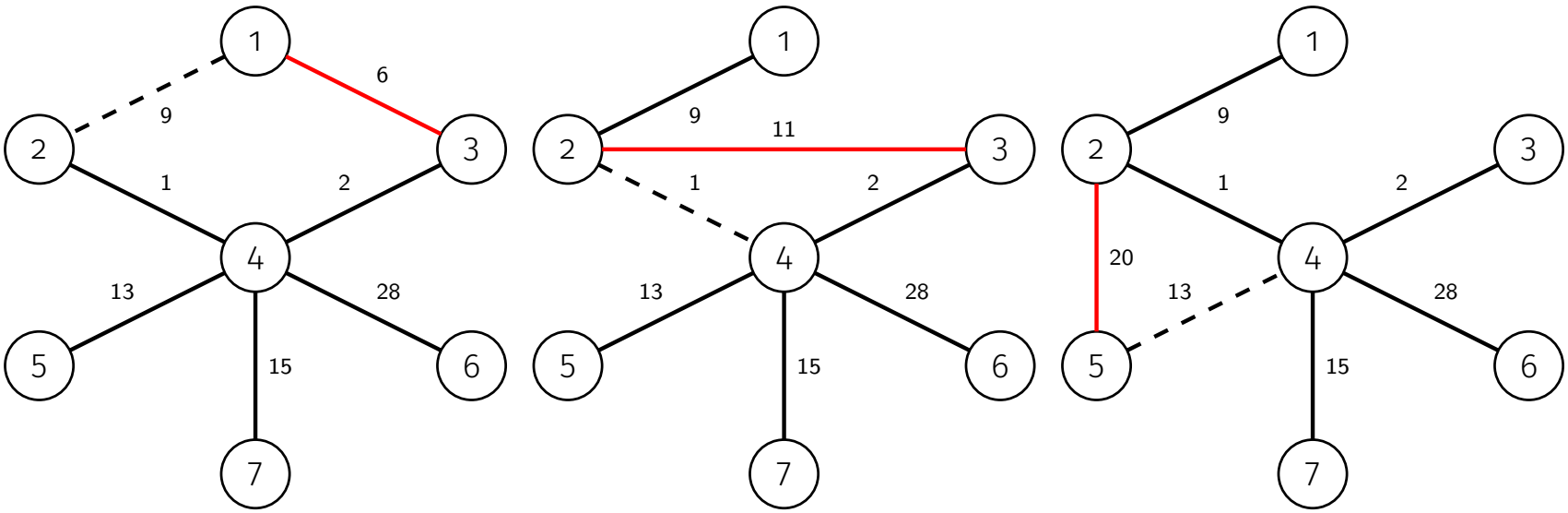
Sol = 68-9+6=65



Sol = 68-9+18=77
Infeasible!

Neighborhood definition

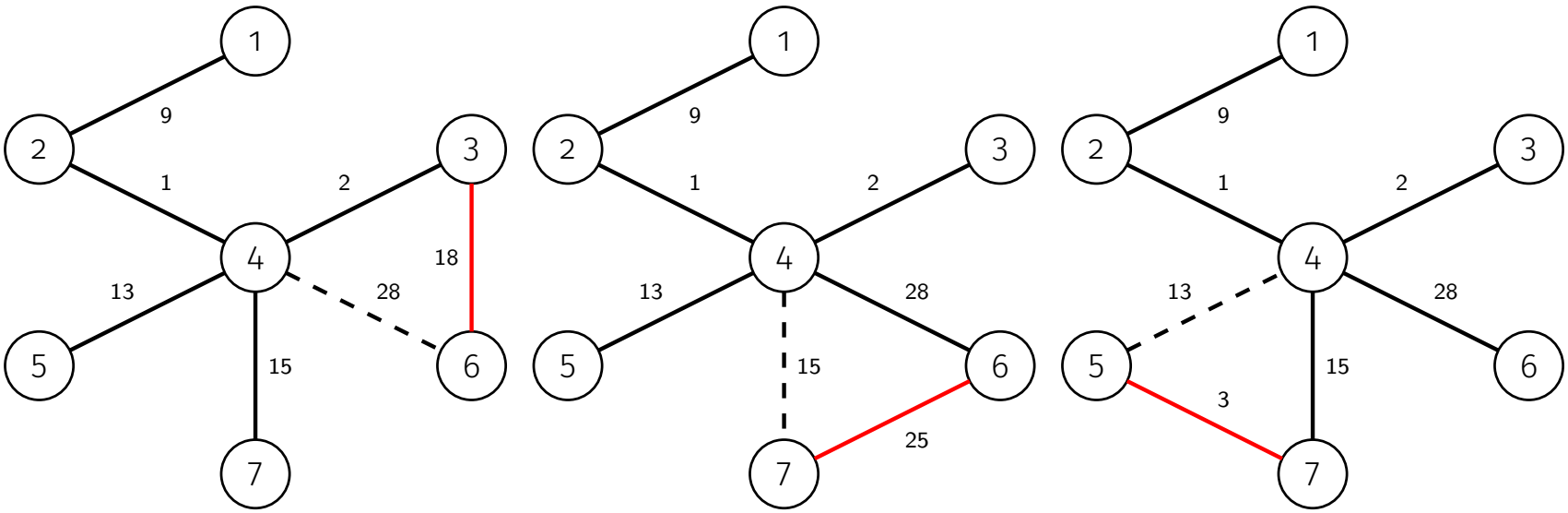
Set of all the (feasible) solutions that can be generated by performing the chosen move to the initial solution.



Sol = 68-9+6=65 Sol = 68-1+11=78 Sol = 68-13+20=75

Neighborhood definition

Set of all the (feasible) solutions that can be generated by performing the chosen move to the initial solution.

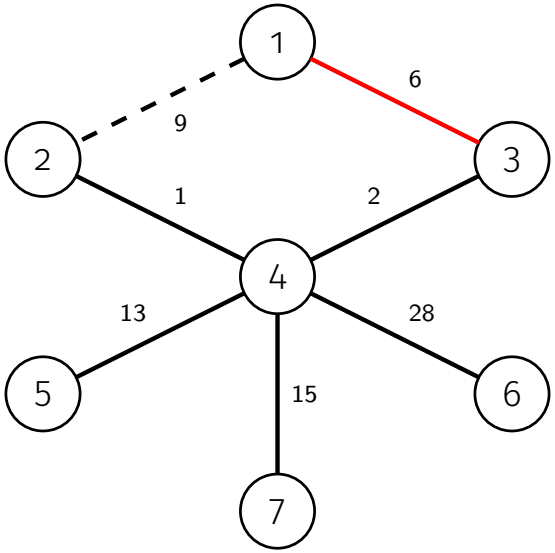


Sol = 68-28+18=58

Sol = 68-15+25=78

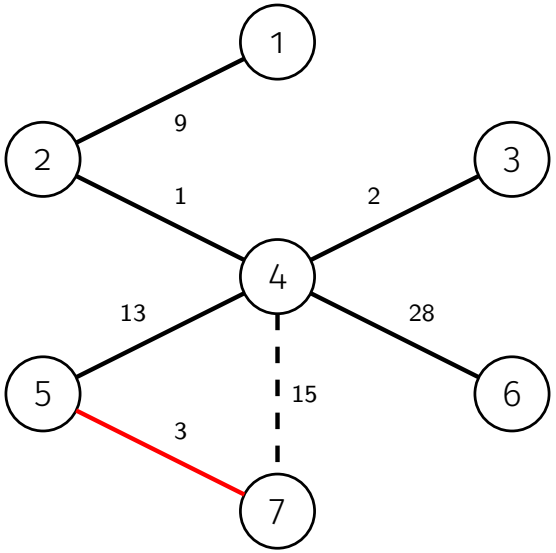
Sol = 68-13+20=75

First Improvement



Sol = 68-9+6=65

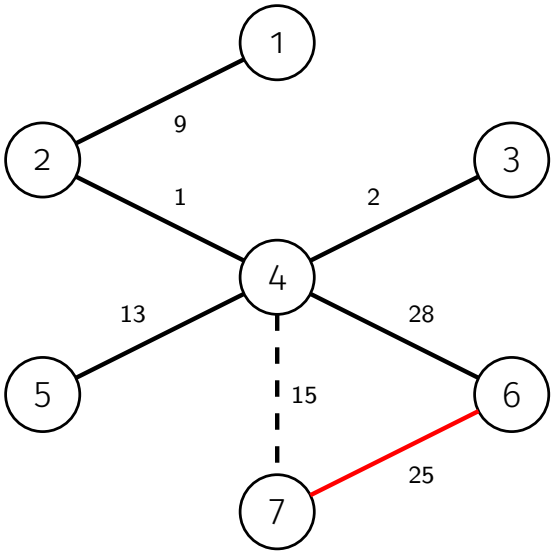
Best Improvement



Sol = 68-15+3=56

Random Move

$$P(Sol_1) = \begin{cases} e^{(Sol - Sol_1)} & Sol_1 > Sol \\ 1 & otherwise \end{cases}$$



Sol_1 = 68-15+25=78

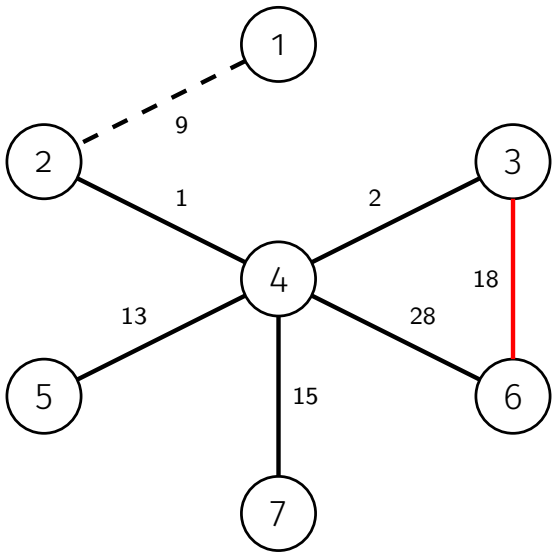
$$P(Sol_1) = e^{(68-78)} = e^{-10}$$

Controlled infeasibility

- We can perform moves that produce infeasible solutions
- Objective function penalty:

$$S^* = S + \alpha * \mathit{numOfIsolatedNodes}$$

- Limit on the number of iterations with an infeasible solution.



One-bit-flip local search

- Requirement: the solution can be represented as a binary vector V

$$V(i) = \begin{cases} 1 & \text{item } i \in \mathit{Solution}; \\ 0 & \text{otherwise.} \end{cases}$$

- Start from an initial solution (possibly randomly generated);
- At each iteration generate all the solutions in which 1 bit is flipped from 0 to 1;
- Choose the best solution and use it as the new incumbent;
- If no solution improves the incumbent, stop. Otherwise: rinse and repeat.

One-bit-flip local search: Knapsack Problem

- Example: Knapsack Problem with 7 items and capacity equal to 100

i	1	2	3	4	5	6	7
w_i	40	50	30	10	10	40	30
p_i	40	60	10	10	3	20	60

- Initial random solution $s_1 = 40$

1	2	3	4	5	6	7
1	0	0	0	0	0	0

- Best neighbor $s_2 = 100$

1	2	3	4	5	6	7
1	1	0	0	0	0	0

One-bit-flip local search: Knapsack Problem

- One obvious problem: local optima.
- At the next iteration we reach a local optimum, $s_3 = 110$:

1	2	3	4	5	6	7
1	1	0	1	0	0	0

- The global optimum is $s_4 = 133$

1	2	3	4	5	6	7
0	1	0	1	1	0	1

- We will never reach s_4, since we will stop after finding s_3.

One-bit-flip local search: Knapsack Problem

How to improve this local search?

- Allow 1-0 flips (coupled with 0-1 flips)
- N-bit-flip local search
  - How many bit we should flip from 0 to 1 at each iteration?
  - How many bit we should flip from 1 to 0 at each iteration?
- Allow worsening moves
  - with a certain probability;
  - for a limited number of iterations.
- What if we can not test all the possible combinations at each iteration? How do we prioritize?
  - Static criterion (first random k moves);
  - Use quality of the solutions/moves encountered during the search.